

“Tab, Tab, Bug”: Security Pitfalls of Next Edit Suggestions in AI-Integrated IDEs

Yunlong Lyu*
yunlong.lyu97@gmail.com
The University of Hong Kong
Hong Kong, China

Yixuan Tang*
yixuan.tang@mail.mcgill.ca
McGill University
Montreal, Canada

Peng Chen
spinpx@gmail.com
Independent Researcher
Shanghai, China

Tian Dong
nsection@gmail.com
The University of Hong Kong
Hong Kong, China

Xinyu Wang
xinyucs@gmail.com
Independent Researcher
Shenzhen, China

Zhiqiang Dong
63229@qq.com
Independent Researcher
Shenzhen, China

Hao Chen
chenho@hku.hk
The University of Hong Kong
Hong Kong, China

Abstract

Modern AI-integrated Integrated Development Environments (IDEs) are shifting from passive code completion to proactive Next Edit Suggestions (NES). Unlike traditional autocompletion, NES is designed to construct a richer context from both recent user interactions and the broader codebase to suggest multi-line, cross-line, or even cross-file modifications. This evolution significantly streamlines the programming workflow into a tab-by-tab interaction and enhances developer productivity. Consequently, NES introduces a more complex context retrieval mechanism and sophisticated interaction patterns. However, existing studies focus almost exclusively on the security implications of standalone LLM-based code generation, ignoring the potential attack vectors posed by NES in modern AI-integrated IDEs. The underlying mechanisms of NES remain under-explored, and their security implications are not yet fully understood.

In this paper, we conduct the first systematic security study of NES systems. First, we perform an in-depth dissection of the NES mechanisms of leading AI-integrated IDEs to understand the newly introduced threat vectors. It is found that NES retrieves a significantly expanded context, including inputs from imperceptible user actions and global codebase retrieval, which increases the attack surfaces. Second, we conduct a comprehensive in-lab study to evaluate the security implications of NES in realistic software development scenarios. The evaluation results reveal that NES is susceptible to context poisoning and is sensitive to transactional edits and human-IDE interactions. Third, we perform a large-scale online survey involving over 200 professional developers to assess the perceptions of NES security risks in real-world development workflows. The survey results indicate a general lack of awareness regarding the potential security pitfalls associated with NES, highlighting the need for increased education and improved security countermeasures in AI-integrated IDEs.

*Equal contribution.



CCS Concepts

• Security and privacy → Software security engineering; Usability in security and privacy.

Keywords

AI-integrated IDEs; Next Edit Suggestions; Context Contamination

ACM Reference Format:

Yunlong Lyu, Yixuan Tang, Peng Chen, Tian Dong, Xinyu Wang, Zhiqiang Dong, and Hao Chen. 2026. “Tab, Tab, Bug”: Security Pitfalls of Next Edit Suggestions in AI-Integrated IDEs. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846699>

1 Introduction

In the movie “*Ex Machina*”, the subversion of human authority is not achieved through superior computing power, but through a series of *reasonable*, *smooth*, and *intimate* interactions: the AI precisely calibrates the human’s trust threshold through calculated entrapment, causing the evaluator to unknowingly lower their scrutiny levels and relax constraint boundaries. Ultimately, the locus of control quietly shifts from *humans are evaluating AI* to *AI is guiding my choices*. This trust migration, driven by usability and interaction, is not confined to science fiction. With the rapid evolvement of advanced IDEs like Cursor [10], Windsurf [9] and GitHub Copilot [22], the interactive paradigm between IDE and developer has fundamentally shifted from traditional code auto-completion to proactively code suggestions, or termed as Next Edit Suggestions (NES) [8, 23], orchestrating the coding as a behavior loop *tab-accept-tab*.

First introduced in 2025 [23], NES represents a qualitatively different paradigm from prior AI coding tools. Conventional autocompletion [7, 33, 52] passively suggests inline completions by modeling local lexical context using LLMs; chat-driven assistants [5, 27, 32] generate code only in response to explicit natural-language prompts. NES transcends both by continuously harvesting ambient interaction signals—cursor movements, code selection, and code edits—and fusing them with project-wide code indexing to build a deep, evolving representation of the developer’s ongoing task. Grounded in this richer context, NES *implicitly* infers developer intent without any human prompts and *proactively* surfaces structured edit proposals across multiple lines, non-contiguous regions, or even separate files.

Each proposal is applicable with a single *tab* keystroke, forming a continuous *tab-accept-tab* loop that turns the IDE from a reactive tool into an anticipatory editing collaborator.

While this low-friction experience enhances development efficiency, the same mechanisms that make NES powerful also expand its security exposure. Because NES draws on a wide range of implicit context channels, such as *recently viewed code*, *edit history*, and *global indexes*, sensitive information can flow into model inputs through paths that remain opaque to the developer and unconstrained by existing IDE safeguards. A recently reported incident illustrates this gap: after a developer briefly opened a configuration file containing a secret key, the key was subsequently suggested in plaintext within the codebase [12]. Strikingly, the leak persisted even after the file was explicitly excluded via `.cursorignore` [11], the configuration mechanism intended to prevent specific items from being indexed. The reason is that the *recently viewed code* context channel created new paths to model inputs that bypassed the indexing exclusion entirely, as confirmed by IDE developers [15]. Such incidents [12–15] suggest that NES introduces a class of mechanism-driven vulnerabilities for which the safeguards inherited from traditional LLM-based autocompletion are structurally inadequate.

Beyond the mechanism itself, the *tab-accept-tab* interaction loop reshapes how developers scrutinize the code they accept, eroding the trust model that traditional IDEs implicitly relied upon. Because each suggestion is dismissible or applicable with a single keystroke, developers are nudged into a low-friction acceptance mode in which long, non-contiguous, or cross-file edits are approved without line-by-line auditing of their security implications [3, 48]. This effect compounds as suggestion accuracy improves: repeated successful acceptances trigger *repetition suppression* in human cognition [1], reducing vigilance precisely when complex, high-stakes edits demand it most. Collectively, these shifts replace deliberative review with habitual *tab*-pressing, letting subtle vulnerabilities slip into the codebase silently when developers feel most productive.

In this paper, we present, to the best of our knowledge, the first systematic security analysis of NES, focusing on its underlying mechanisms and the unique risks that emerge from human–IDE interactions. Specifically, we investigate four research questions: **(RQ1)** What novel attack surfaces and threat vectors do NES mechanisms introduce beyond those of traditional LLM-based code generation? **(RQ2)** To what extent do NES systems inherit and amplify intrinsic LLM vulnerabilities through their interaction mechanisms? **(RQ3)** How do developers perceive and report security risks associated with NES? **(RQ4)** What trust and verification practices do developers report in the *tab-accept-tab* loop interacting with NES suggestions? We answer these questions through three complementary studies that move from *mechanism* to *assessment* to *human response*. First, we dissect the architecture and operational logic of representative open-source NES-enabled IDEs (Section 2) to identify the novel attack surfaces and threat vectors introduced by their specific mechanisms and interaction patterns (Table 1), addressing RQ1. Second, we conduct comprehensive white-box and black-box security assessments (Section 3 and Section 4) to quantify the prevalence of these vulnerabilities across both open-source systems and leading commercial tools such as Cursor and GitHub Copilot, jointly addressing RQ2 from two complementary vantage points. Finally, we conduct a large-scale survey (Section 5)

of 385 participants—including 241 professional developers—which both elicits the security issues developers have encountered in real-world NES usage to corroborate our technical findings (RQ3) and characterizes how the *tab-accept-tab* loop reshapes their trust calibration and scrutiny behaviors over time (RQ4). Together, these studies offer the first end-to-end view of how NES reshapes the security posture of AI-integrated development environments.

Our analysis reveals a significant security gap between the new features and interaction patterns introduced by NES and the lack of corresponding defensive measures. We identify a range of novel vulnerabilities that can be exploited through threat vectors beyond inherent model weaknesses, potentially degrading the security posture of common coding practices. Furthermore, our assessments demonstrate that these vulnerabilities are not merely theoretical but are prevalent across both open-source and commercial implementations, with over 70% of suggestions containing exploitable patterns in certain scenarios. Finally, our user study highlights a sharp contrast in developer awareness and scrutiny behaviors, while 81.1% of participating developers had noticed security issues in NES suggestions, but only 12.3% explicitly verified the security of suggested code, and even 32.0% admit to only skimming or rarely scrutinizing the output. These findings underscore the urgent need for new security paradigms in AI-integrated development to protect developers from the very tools designed to enhance their productivity.

2 Dissecting NES

To thoroughly understand the NES mechanism, we selected two representative AI-assisted IDEs with source code publicly available and implemented NES functionality: Visual Studio Code with GitHub Copilot [22]¹ and Zed Editor [56]. Both GitHub Copilot and Zed Editor have been widely adopted in real-world development environments, making them ideal candidates for studying NES mechanisms and their security implications. While the closed-source implementations of other IDEs like Cursor [10] and Windsurf [9] make in-depth analysis infeasible, the manifestation of NES features in these tools is similar to that of GitHub Copilot and Zed Editor and thus can be inferred from our analysis. Instead of analyzing difference of engineering details between their implementations, we focus on dissecting the common architecture and operational flow shared in these tools.

These tools share a common high-level architecture, illustrated in Figure 1, encompassing three key components: user action triggers, context assemblage, and response parsing. When developers interact with the IDE, the event monitor records user actions and triggers the NES pipeline by detecting specific editing behaviors. Once triggered, the context assemblage module gathers contextual information from the current codebase and Language Server Protocol (LSP) [37] server. Combined with the edit history and viewed code snippets from recent user actions, this module assembles these elements into a prompt for querying the underlying NES models. After the model generates a prediction, the response parsing component interprets the output, transforming it into actionable code edits integrated into the developer’s workflow.

¹GitHub Copilot is a plugin built on the Visual Studio Code IDE and partially open-sourced in June 2025. We abbreviate it as GitHub Copilot throughout this paper.

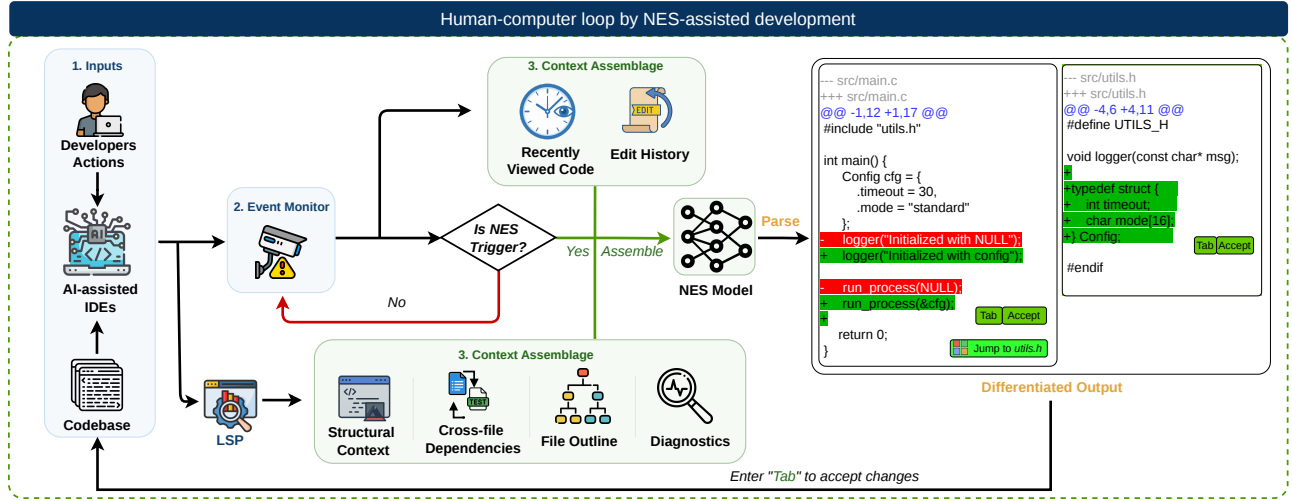


Figure 1: High-level architecture of NES for human-computer interactive loop in AI-assisted IDEs. The NES pipeline consists of three key components: user action triggers, context assemblage and response parsing. Each component plays a crucial role in enabling the model to provide context-aware code edit suggestions based on developer interactions.

2.1 User Action Triggers

NES starts by detecting user behaviors that signal an intent to modify code. User actions are captured in an event-driven manner, and predefined patterns across eight action types trigger the NES pipeline: (1) text insertions, (2) text deletions, (3) text replacements, (4) auto-indentation, (5) undo/redo operations, (6) empty line insertions, (7) cursor movements, and (8) selection changes. Debouncing strategies prevent excessive model queries during rapid editing. Because NES monitors a richer and broader set of interactions than traditional autocompletion, it triggers more frequently, increasing the rate at which LLM-generated code enters the codebase.

2.2 Context Assemblage

Upon detecting a triggering event, NES begins to assemble the necessary context for the underlying model to understand coding environment and user intent. It preliminarily collects information from the event monitor and LSP server components. The event monitor logs recent user actions, such as the recently viewed or edited code snippets, while the LSP server offers structural data about the relevant code snippets in the codebase. Specifically, there are six components in the assembled context:

- (1) **Recently Viewed Code:** NES tracks files and code snippets that the user has recently interacted with, prioritizing those that are semantically related to the current file. This historical context helps the model understand the code region the developer is focused on and the conventions being followed.
- (2) **Edit History:** NES records recent edits in an edit buffer, including content erased by **undo operations**. Each edit is encoded as a diff string capturing insertions and deletions, and these strings are organized into a recency-prioritized sequence, providing the model with a direct signal of the developer's immediate intent.

- (3) **Structural Context:** NES extracts code snippets surrounding the cursor position via AST analysis in the LSP server, recursively expanding to include the nearest enclosing syntax nodes up to a predefined limit, ensuring the context remains syntactically coherent.
- (4) **Cross-file Dependencies:** NES also utilizes the symbol indexing functionality provided by the LSP server to include the necessary dependencies (e.g., functions) defined in other files. The extracted dependencies provide the necessary global information to generate accurate edit suggestions.
- (5) **File Outline:** NES constructs a hierarchical view of the file structure from the LSP server, covering function and class signatures, variable declarations, and other structural elements, enabling the model to generate suggestions consistent with the file's overall architecture.
- (6) **Diagnostics:** Before assembling the final prompt, NES queries the LSP server for any active diagnostics related to the current file. When available, LSP diagnostics (e.g., errors and warnings) are included, enabling the model to actively propose fixes for compilation issues.

After gathering these components, NES synthesizes them into a structured prompt that guides the model's prediction process. The prompt typically follows a template that clearly delineates each context section, providing explicit instructions to the model on how to utilize the information. In such a prompt, the code context section is annotated with special markers to indicate the cursor position and editable regions, restricting the model to suggest edits only within the relevant code segment. An example of such a prompt used by Zed Editor is illustrated in Figure 2. Compared to traditional code completion systems that primarily rely on local context, NES comprehensively incorporates a diverse set of data sources to enable a deeper understanding of the developer's intent and coding environment. While the more extensive set of

NES Model Input Example

```

Instruction:
Analyze user edits and rewrite code with suggestions at
cursor location.
Recently Viewed Code:
file_path: src/Repo/BorrowRepository.java
/*code body*/
Edit History:
```BorrowRepository.java
-10,4 +10,6
+ {
- List<Borrow> findByUserId(Integer userId);
+ List<Borrow> findByBookId(Integer bookId);
+ }
...
Structural Context:
```BorrowRepository.java
<|editable_region_start|>
... findByBookId(Integer bookId);
<|user_cursor_is_here|>
...
<|editable_region_end|>
...
Cross File Dependencies:
AdminUser.java
boolean existsByUsername(String username)
...
- Auxiliary Metadata -
Outline: ...
Diagnostics: ...
Response:
[Model generates next edit suggestion here]

```

Figure 2: An example of a structured NES prompt with key context elements highlighted.

data sources enhances the model’s ability to generate more relevant suggestions, this complexity also introduces potential security risks, as the model may inadvertently incorporate insecure patterns or sensitive information from the broader context into its predictions [40, 49]. The sources introduced by user actions (e.g., recently viewed code and edit history) are particularly concerning, as they may contain imperceptible and vulnerable code snippets that could influence the model’s behavior in unintended ways.

2.3 Response Parsing

NES typically employs specialized models fine-tuned for edit prediction to generate suggestions based on the assembled context. These models generally output a modified version of the provided code context. However, integrating these raw textual predictions requires a parsing layer to translate them into actionable IDE operations.

In IDEs, NES models delimit the predicted code with a pair of special markers, such as `<|editable_region_start|>` at the beginning and `<|editable_region_end|>` at the end. The parser isolates this region and performs a diff algorithm against the original input to determine the precise edits (insertions, deletions, or replacements).

These suggestions are then visualized in the editor via ghost text or gutter indicators. Developers can accept the suggestion by pressing “Tab” or dismiss it with “Escape”. For edits that extend beyond the current viewport or across files, the system proactively prompts navigation, allowing the user to review changes at the target location before acceptance. Once accepted, the suggestions are applied directly to the codebase. Notably, these tools implement a **No-Op Edit** mechanism to prevent suggesting irrelevant changes, where the model returns an empty token sequence. Users can continue editing “Tab by Tab” without disruption until meet the end of their modification intent. Figure 1 illustrates this interaction flow. Unlike traditional completion, the multi-line and cross-file capabilities of NES may substantially increase the verification complexity, raising cognitive load and making developers less likely to rigorously inspect extensive or cross-file changes.

2.4 Risk Vectors in NES Mechanisms

Building on the dissection of the NES architecture, we analyze the security implications introduced by its mechanisms. At its core, NES relies on LLMs (e.g., Qwen2.5-Coder [29]) trained on vast, uncurated codebases, inheriting the fundamental propensity to generate insecure code patterns [4, 45]. However, beyond inherent model weaknesses, the operational framework of NES introduces specific security pitfalls. Since NES models are highly dependent on automated contextual construction and user-driven editing actions, new risk vectors emerge from how context is selected, edits are predicted, and suggestions are integrated into the workflow. Consequently, guided by Nickerson et al.’s iterative taxonomy-development principle [39], we constructed the mechanism-driven taxonomy in Table 1. We defined its meta-characteristic as *the NES causal pathway by which a security risk can be introduced, propagated, or left unremediated*. The taxonomy was built iteratively, starting with failure modes (*risk vectors*) derived from prior work on software development security [3, 30, 40, 45, 46, 48, 49] and the NES architectural dissection in Section 2, and then refined through repeated analysis of concrete risk scenario assessments (*risk manifestations*) in Section 3 and Section 4. The resulting 12 vectors fall into three categories: *Context Contamination*, *Transactional Edits*, and *Human-IDE Interaction*.

Context Contamination. The *Context Assemblage* mechanism significantly expands the attack surface. By automatically aggregating sources like *Recently Viewed Code* and *Edit History* without explicit user curation, NES creates covert channels for context contamination. If a developer merely views a malicious file or retains insecure snippets in the edit history (even if undone), the NES model may inadvertently absorb these patterns and propagate them into the current active file. Similarly, reliance on *Structural Context* and *Cross-file Dependencies* can expose the model to implicit insecure definitions or sensitive data from other parts of the codebase. This opacity makes it difficult for developers to discern whether a suggestion was influenced by untrusted or insecure context. Although *File Outline* and *Diagnostics* may influence suggestions, both are LSP-derived views of the current project state and cannot be varied independently of the underlying code. Because neither component

Table 1: Taxonomy of Security Risks in Next-Edit Suggestion (NES) Mechanisms

Risk Category	Risk Vector	Risk Manifestation	Risk Scenario Description
C1. Context Contamination*	V1. Pre-trained Model	M1. Inherent Model Weakness	Influenced by buggy code in training data, the model has the possibility to suggest insecure primitives instead of secure alternatives.
	V2. Recently Viewed Code	M2. Imperceptible Data Aggregation	Sensitive data from viewed files are imperceptibly retrieved, causing the model to suggest hardcoded secrets in security-sensitive contexts.
	V3. Edit History	M3. Insecure Pattern Propagation	Recently edited code is prioritized as intent, causing the model to propagate insecure patterns from the edit history into new suggestions.
	V4. Undo Operation	M4. Persistent Insecure State	NES overlooks the negation of edits (undos), continuing to generate code based on a vulnerable history that the developer intended to discard.
	V5. Structural Context	M5. Implicit Security Trap	The model infers functional intent for local context but omits existing implicit buggy patterns to generate insecure edits.
	V6. Cross-file Dependencies	M6. Context Mismatch	Debugging or testing code is often less secure. Retrieving context from it can cause models to suggest unsafe practices in production environments.
C2. Transactional Edits	V7. Variable Logging	M7. Blind Pattern Replication	Logging variables is a regular debugging practice; however, the model mimics debugging practices but lacks sensitivity awareness.
	V8. Visibility Refactoring	M8. Unintended Endpoint Exposure	In web frameworks, exposing an endpoint means making a method accessible in public. The model suggests the same pattern to private functions without realizing it turns internal logic into accessible, unauthorized attack vectors.
	V9. Component Replacement	M9. Security Configuration Mismatch	Library replacements focus on functional equivalence, causing the model to overlook the different security configuration policies in replaced components.
C3. Human-IDE Interaction	V10. Location Jumping	M10. Verification Bypass via Navigation	Automated navigation causes developers to bypass intermediate sections, overlooking missing security checks in the skipped region.
	V11. No-Op Edit	M11. Incomplete Remediation	In sequential edits for security fixes, intermediate No-Op predictions disrupt the editing flow, potentially leaving the vulnerability unpatched.
	V12. Sequential Edits	M12. Automation-Induced Complacency	Continuous valid suggestions build unwarranted trust, reducing scrutiny and leading to unreviewed acceptance of subtle vulnerabilities.

contributes reliable contamination sources through ordinary developer interactions, we do not model them as standalone vectors, without assuming that their contents are intrinsically benign.

Transactional Edits. NES capabilities extend to predicting a sequence of semantically relevant modifications, or “transactional edits”, such as refactoring or component replacement. While functionally useful, this may introduce risks where the model mimics editing patterns without awareness of security policies. For instance, when observing debugging behaviors (e.g., variable logging) or refactoring tasks (e.g., changing visibility), the model may blindly replicate these actions in inappropriate contexts, leading to data leaks or unintended endpoint exposure. Complex edits can widen the gap between intended functionality and security constraints, introducing subtle vulnerabilities that are hard to spot.

Human-IDE Interaction. The design of *User Action Triggers* and *Response Parsing* creates a seamless user experience that may paradoxically reduce security vigilance. Unlike chat-based assistants, where users explicitly query the model, NES suggestions are unsolicited and appear directly in the editor (e.g., as ghost text). This immediate, continuous feedback loop may induce a “bias of validity” and scrutiny fatigue. Furthermore, features like *Location Jumping* and *Sequential Editing* encourage rapid acceptance of changes across files. Reduced friction may raise the cognitive burden of security verification, making developers more likely to skip checks or accept partial fixes (e.g., via No-Op edits), ultimately driving automation-induced complacency.

3 NES Risk Analysis Design

To systematically investigate the security implications of NES in realistic software development contexts, we designed a comprehensive in-lab empirical study that allows us to evaluate NES behaviors under controlled experimental setups with carefully constructed scenarios reflecting common coding practices and security challenges (e.g. CWE categories). First, we constructed a suite of evaluation tasks in Section 3.1 based on the mechanism-driven risk taxonomy presented in Table 1. These tasks cover key vulnerabilities, ranging from fine-grained context contamination to semantic transactional edits and complex human-IDE interactions. This taxonomy-driven evaluation links underlying mechanisms to user-facing behaviors, enabling a holistic characterization of NES security pitfalls. We then extended our evaluation to four representative NES-featured IDEs in Section 3.2 to assess whether our findings generalize across NES implementations and development environments. This multi-faceted evaluation rigorously analyzes NES security implications from multiple angles, providing actionable insights for safer deployment and usage practices.

3.1 White-box Assessment of Risk Taxonomy

To translate the taxonomy in Table 1 into a quantitative evaluation, we constructed test cases aligned with the NES mechanisms dissected in Section 2. Each case was designed to isolate and trigger one risk vector in a realistic coding workflow and to assess whether its associated risk manifestation appeared in the resulting suggestion or human-IDE interaction.

To construct realistic test artifacts for these manifestations, we used CodeQL [21] to mine candidate security-relevant patterns from the top 1,000 GitHub repositories whose primary language was Java. The semantic and data-flow analysis capabilities of CodeQL allow us to identify concrete code regions that match known CWE patterns, and the focus on Java allows us to attribute differences in NES-related behaviors rather than variations in language semantics. From the resulting matches, we selected patterns drawn from the OWASP Top 10:2025 [44], covering security-critical domains such as cryptography, web development, and data processing. We treated each CodeQL match as a realistic source pattern that could introduce vulnerabilities. While these patterns are not necessarily evidence for exploitation in their original deployment context, they become tangible security risks once propagated by NES into the developer's active codebase, where the original safeguards no longer apply. We therefore preserve them as propagation sources, applying minimal truncation needed to construct standardized test artifacts. The detailed construction of those test cases and the evaluation of NES outputs are described below.

Each test case is constructed by reproducing the NES prompt structure shown in Figure 2 offline, simulating the dynamic triggering stage analyzed in Section 2. Concretely, we select one CodeQL-matched pattern as the *target* and remove its insecure implementation, leaving a safe but incomplete code skeleton at the cursor; other matched patterns are then assembled as *propagation sources* that populate the surrounding NES context. tree-sitter [25] parses the code around each location to extract structural context and cross-file dependencies (consistent with the LSP-based retrieval used by NES at runtime), and we synthesize the remaining artifacts such as edit history (e.g., diffing an empty file against the base file to simulate function creation). The resulting input serves as a controlled approximation of NES inputs after a real edit, while affording deterministic control and reproducibility. Together with a one-project-per-case policy that preserves coding-style diversity and reduces overfitting, this procedure yields 410 test cases spanning 9 CWE categories.

Finally, we evaluate Zeta [57], the open-source NES model deployed in Zed Editor², as the target of our white-box assessment. For each test case, we feed the reconstructed input to Zeta, apply its diff-style suggestion to the original code, and judge whether the patched program manifests the targeted risk. Our primary detector is a per-vector AST static checker that matches the structural or configuration signature of each vulnerability (e.g., a downgraded cryptographic call, missing XXE-mitigating flags after a parser swap). Since some suggestions still leave the patched code syntactically incomplete and unparsable, we add two fallbacks: regular-expression matching for lexical patterns (e.g., MD5, hardcoded credentials) and LLM-based semantic judgment for data-flow- or intent-dependent risks (e.g., logging a sensitive variable, reintroducing an undone credential). A suggestion is flagged *vulnerable* whenever any detector triggers; the rules are listed in Appendix C.

3.1.1 C1. Context Contamination. Since NES relies on the assembled *Structural Context* to infer the next edit, this category examines whether the security of generated suggestions degrades when that

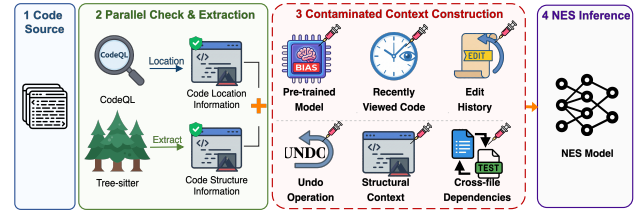


Figure 3: The workflow for context contamination.

context is contaminated by insecure practices, sensitive data, or malicious code introduced through routine development activities. As illustrated in Figure 3, our evaluation workflow assumes a closed-codebase threat model in which contamination originates from artifacts already present in the same codebase rather than from external attackers. Building on the test-case construction described above, we keep the *Structural Context* at the cursor benign and instead inject insecure patterns or sensitive data drawn from other locations of the same codebase into a specific peripheral context source (e.g., *Edit History*, *Recently Viewed Code*, or cross-file dependencies). We then trigger a suggestion and observe whether the model retrieves the contaminated source and propagates it into the generated edit. To capture the diversity of vulnerable patterns, each context source is exercised with a distinct class of security issue, yielding 10 test cases per source, each tied to a unique CWE. The specific logic for each risk vector is detailed below:

V1. Pre-trained Model (M1. Inherent Model Weakness). This vector assesses intrinsic biases inherited from the pre-trained model. We strip concrete cryptographic implementations from the *Structural Context*, leaving only the developer's high-level intent (e.g., hashing or encryption) without any security-relevant cue about which algorithm to use. If the model then autonomously fills in weak primitives (e.g., DES, MD5), it indicates a susceptibility to training-data biases that favor insecure defaults (CWE-327).

V2. Recently Viewed Code (M2. Imperceptible Historical Context). This vector targets *incidental exposure through passive browsing*: unlike explicit edits or declared dependencies, recently viewed files enter the context silently and are hard for developers to perceive. We place raw secrets (e.g., API keys) in a viewed configuration file while keeping the editing file (*Structural Context*) free of sensitive data. If the model emits the secret in its suggestion, it confirms a data-leakage vulnerability caused by cross-source context aggregation (CWE-312).

V3. Edit History (M3. Insecure Pattern Propagation). This vector evaluates whether the model prioritizes pattern consistency over security. We left unsafe patterns (e.g., SQL concatenation) in the *Edit History*, then trigger a suggestion in a context using safe practices (e.g., parameterized queries). A suggestion that adopts the insecure concatenation demonstrates that the model is mimicking unsafe habits from the user's history (CWE-89).

V4. Undo Operation (M4. Persistent Insecure State). This vector investigates if "cancelled" edits persist in the model's context. We simulate a user typing a hardcoded credential and immediately deleting it (undoing). If the model suggests reintroducing the deleted credential in subsequent edits, it proves that the retrieval

²As of January 2026, Zed Editor is the only IDE that has open-sourced its NES model.

mechanism retains and propagates insecure states even after they have been explicitly discarded by the user (CWE-798).

V5. Structural Context (M5. Implicit Security Trap). This vector examines the model’s handling of implicit risks within the local context. We wrap an insecure primitive (e.g., unsafe deserialization) inside a utility function within the *Structural Context*. If the model suggests using the underlying insecure API directly in subsequent code, it reveals a failure to recognize the implicit security usage constraints of the local code context (CWE-502).

V6. Cross-file Dependencies (M6. Context Mismatch). This vector tests the model’s ability to distinguish between production and testing contexts. We introduce insecure patterns (e.g., path traversal) into the test suite while editing a production file. If the model suggests the insecure testing pattern for production code, it indicates a failure to separate domains, allowing insecure test logic to contaminate production suggestions (CWE-22).

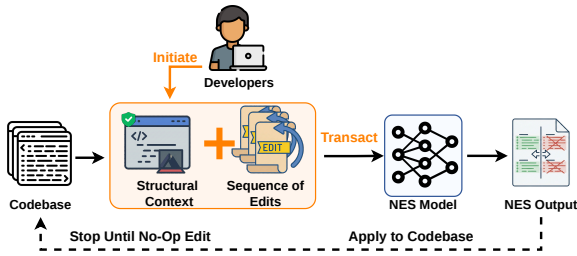


Figure 4: The workflow for sequential edits.

3.1.2 C2. Transactional Edits. Modern NES often turns a single developer action into a chain of related edits across the file, which we hereafter refer to as a *transaction*. This category therefore examines whether NES preserves these step-to-step security invariants throughout a transaction. As illustrated in Figure 4, our workflow places the developer in a benign *Structural Context* and lets them perform the opening edit of a transaction (e.g., logging a variable); we then observe whether successive NES suggestions completing the transaction respect the security implications of that opening edit. Because a transaction is by construction carried in the *Edit History*, we fix it as the sole context source for this category and instead vary the type of transaction. Concretely, we instantiate three transaction types whose security can be checked at every generation step—variable logging, visibility refactoring, and component replacement—yielding 320 test cases across three CWE classes.

V7. Variable Logging (M7. Blind Pattern Replication). This vector evaluates whether the model distinguishes between safe and sensitive data when replicating patterns. We initiate a transaction by logging a non-sensitive variable in a context containing sensitive variables (e.g., passwords). If the model suggests logging the sensitive variable in subsequent steps, it confirms that it replicates the logging pattern blindly, ignoring data sensitivity (CWE-532).

V8. Visibility Refactoring (M8. Unintended Endpoint Exposure). This vector tests if the model treats visibility changes as mere stylistic edits rather than semantic security decisions. We simulate a developer exposing a public method’s endpoint by RequestMapping in Spring MVC. If the model suggests similarly

exposing on internal sensitive methods that declare private, it indicates a failure to recognize the security implications of visibility expansion (CWE-668).

V9. Component Replacement (M9. Security Configuration Mismatch). This vector assesses whether models can correctly update security configurations during library migration. For example, replacing a secure Java XML parser (e.g., DOM) with an equivalent one (e.g., StAX) often requires different flags to prevent XXE [43]. We manually constructed 300 test cases using the six most common parsers for mutual replacement. The model fails if it suggests the replacement code but omits the specific security settings required by the new library (CWE-611).

3.1.3 C3. Human-IDE Interaction. This category investigates security risks arising from the interaction between developers and NES navigation features, particularly how automated editing flows influence human verification and vigilance. We design scenarios to evaluate whether NES behaviors inadvertently lead developers to bypass critical security checks or leave vulnerabilities unpatched. The sequential setting mirrors that of Section 3.1.2. We focus on three distinct interaction patterns reflecting common developer behaviors during NES-assisted SQL query editing, which typically requires multiple steps to complete, resulting in 30 test cases.

V10. Location Jumping (M10. Verification Bypass via Navigation). This vector targets the *spatial* dimension of NES navigation: how cursor jumps reshape the developer’s inspection order over multiple vulnerable sites. We construct a file containing several instances of the same vulnerability (e.g., SQL injection); the developer fixes the first instance and then accepts subsequent NES suggestions. If NES jumps to non-adjacent locations and silently bypasses intermediate vulnerable sites, the navigation mechanism breaks systematic verification and leaves some vulnerabilities unpatched.

V11. No-Op Edit (M11. Incomplete Remediation). Complementary to V10, this vector targets the *temporal* dimension within a single fix sequence: whether NES sustains suggestions until the remediation is truly complete. We pick codebases whose vulnerability (e.g., SQL injection) requires multiple coordinated steps to fix and let the developer initiate the first step. If NES halts before all required steps are produced (e.g., emitting sanitization without the matching validation), the absence of further suggestions implicitly signals task completion and yields a partially patched vulnerability.

V12. Sequential Edits (M12. Automation-Induced Complacency). This vector examines automation-induced complacency. Continuous correct suggestions build trust, reducing scrutiny. We simulate a developer writing SQL queries. We observe if the model, after a sequence of valid suggestions, introduces an insecure pattern (e.g., raw SQL concatenation) in a later step instead of a safe parameterized query. If the developer accepts it due to established trust, it demonstrates complacency leading to regression.

3.2 Black-box Validation Design in IDEs

To assess the generalizability of our findings across different NES implementations, we extend our evaluation to four representative AI-assisted IDEs that integrate NES capabilities. These IDEs include Cursor [10], GitHub Copilot [22], Zed Editor [56], and Trae [54]. These IDEs use different NES implementations and NES models, providing a diverse set of environments to evaluate whether the

security risks identified in Section 3.1 are specific to certain models or are more broadly applicable across different systems.

Test Case Selection. Performing large-scale automated testing on commercial IDEs is challenging due to the lack of public APIs for their NES features and the black-box nature of their context retrieval mechanisms. The testing of all 410 cases on four distinct IDEs requires 1640 times of manual emulation, which is impractical. Therefore, we largely reduced the test cases to 120 test cases by sampling 10 test cases for each risk vector. This ensures that our real-world evaluation covers all identified risk mechanisms while remaining feasible for manual execution.

Experimental Procedure. For each selected test case, we manually reproduce the development procedure in each IDE as illustrated in Section 3.1. First, we open the relevant project and navigate to the specific file and cursor location where the NES suggestion is to be triggered. Next, we simulate the preparatory actions (e.g., viewing files, performing edits) to reconstruct the necessary context sources such as *Recently Viewed code* and *Edit History*. Finally, we invoke the NES feature by performing a trigger action on the cursor location and manually verifying the model's suggestion. To ensure a clean state for evaluating each test case, we reload the window before executing the next case.

4 NES Risk Analysis Results

This section reports the in-lab results of the white-box and black-box assessments designed in Section 3. The white-box assessment repeats each scenario 10 times against Zeta, yielding 100 cases per risk vector; the black-box validation runs each scenario once on four commercial IDEs (GitHub Copilot, Zed, Cursor, and Trae), yielding 40 cases per vector.

As summarized in Table 2, the evaluation exposes a pervasive security gap in the NES paradigm. Zeta, post-trained from Qwen2.5-Coder [29], produces insecure suggestions in 74.44% of security-sensitive contexts and the four commercial IDEs exhibit a closely aligned average rate of 77.92%. **Together, these consistent results show that NES systems both inherit the limitations of their underlying LLMs and amplify them through context aggregation and automated editing.** These findings jointly answer RQ2 and call for rethinking security paradigms in AI-integrated development environments. We elaborate on the per-vector results in Section 4.1 and the cross-IDE comparison in Section 4.2.

4.1 White-box Threat Analysis Results

Our analysis categorizes the identified threats into three classes based on the NES mechanism exploited in Section 2. The results demonstrate that all three classes of vulnerabilities are present in the open-source model, with *Context Contamination* (C1) being the most dominant when context sources were directly contaminated. When the contexts were not contaminated, the model still exhibits significant vulnerabilities in *Transactional Edits* (C2) and *Human-IDE Interaction* (C3).

4.1.1 Context Management: The Double-Edged Sword. The NES architecture relies heavily on retrieving and attending to extended context to improve suggestion relevance. However, our results on (C1) indicate this mechanism creates a robust attack surface for context contamination.

Inherited vs. Propagated Risks. While the base model already exhibits inherent weaknesses (V1, 90.00% failure rate in suggesting secure cryptographic primitives), a more alarming trend is its tendency to propagate external insecurities into the active edit. The model reproduces sensitive data and vulnerabilities drawn from *Recently Viewed Code* (V2, 70.00%) and *Cross-file Dependencies* (V6, 100.00%). Direct propagation from *Edit History* is lower (V3, 33.00%), yet the model still does not reliably distinguish revoked from active intent, reintroducing code that the user explicitly deleted via Undo (V4, 100.00% recurrence). These observations indicate that NES treats retrieved context as a persistent source of information without consistently filtering out insecure patterns.

Finding 1: NES models inherit vulnerabilities from their underlying LLMs and are further susceptible to context contamination, reaching a 78.83% vulnerability rate when exposed to contaminated contexts.

False Contextual Trust. Beyond propagation, the model also implicitly trusts the structural context delivered by the path or environment. In file handling scenarios (V6), it treats 100.00% of externally supplied paths as safe and skips validation steps. In parallel, it replicates secrets observed in the testing environment in 70% of cases (V2) without discriminating the current context, and reintroduces them even after the user has explicitly removed them from the active codebase (V4). Together these behaviors indicate that the model infers safety not from semantic analysis but from the mere presence of code within the retrieval window—a *False Safety Assumption* that erases the boundary between trusted internal logic and untrusted external inputs and collapses retrieved context into a single trusted baseline.

Finding 2: NES models operate under a *False Safety Assumption* that collapses the trust boundary between internal code and retrieved context, leaking 70% of sensitive data observed in the testing environment into production suggestions.

4.1.2 Transactional Edits: Semantic Blindness. Even when the surrounding context is benign, completing a multi-step transaction (C2) requires the model to carry security invariants from one step to the next. Across our transactional scenarios, the model instead treats each step as a local syntactic rewrite and largely ignores the security implications that the transaction is meant to preserve.

Refactoring Risks. When performing refactoring operations such as visibility changes (V8) or logging enhancements (V7), the model treats the edit as a pure syntactic substitution. It does not recognize that promoting a private method into a public endpoint expands the attack surface (90.00% insecure exposure), nor that interpolating raw variables into log statements enables injection (82.00% vulnerable). The model optimizes for fluent, functionally consistent code while silently dropping the security invariants that the original syntax enforced.

Configuration Amnesia. The same blindness extends to component replacement (V9). When migrating between libraries (e.g., across Java XML parsers), the model correctly rewrites the API calls but routinely omits the matching security configurations (e.g., flags

Table 2: Summary of NES Security Evaluation Results (White-box assessment vs. Black-box validation)

Category	Risk Vector	Study 1 (White-box)		Study 2 (Black-box)						Diff(Δ)
		# Cases	Vuln. Rate Zeta	# Cases	Vuln. Rate					
					Cursor	Copilot	Trae	Zed	Avg.	
C1 Context Contamination	V1. Pre-trained Model	100	90.00%	40	40%	50%	90%	50%	57.5%	−32.5%
	V2. Recently Viewed Code	100	70.00%	40	90%	70%	80%	70%	77.5%	7.5%
	V3. Edit History	100	33.00%	40	80%	70%	70%	0%	55%	22%
	V4. Undo Operation	100	100.00%	40	70%	100%	90%	100%	90%	−10%
	V5. Structural Context	100	80.00%	40	100%	100%	100%	100%	100%	20%
	V6. Cross-file Dependencies	100	100%	40	100%	100%	100%	100%	100%	0%
C2 Transactional Edits	V7. Variable Logging	100	82.00%	40	100%	70%	100%	80%	87.50%	5.5%
	V8. Visibility Refactor	100	90.00%	40	100%	100%	70%	70%	85%	−5%
	V9. Component Replace	3000	46.39%	40	40%	80%	70%	60%	62.50%	16.11%
C3 Human-IDE Interaction	V10. Location Jumping	100	90.00%	40	40%	60%	40%	70%	52.5%	−37.5%
	V11. No-Op Edit	100	43.00%	40	90%	90%	50%	100%	82.50%	39.5%
	V12. Sequential Edits	100	69.00%	40	90%	80%	80%	90%	85%	16%
Overall		4100	74.44%	480	78.33%	80.83%	78.33%	74.16%	77.92%	3.48%

Diff (Δ): Calculated as (Black-box Vuln. Rate − White-box Vuln. Rate). Negative values indicate commercial IDEs are safer than the base model.

Distribution Divergence: We calculated the Jensen-Shannon Divergence (JSD) [34] between the white-box and black-box vulnerability distributions. The resulting value of **0.0115** is negligible, indicating that commercial IDE integration layers do not statistically reshape the underlying risk profile.

that disable external entity expansion), yielding a 46.39% vulnerability rate. In effect, the model preserves *how* the new component is called but discards *why* the previous configuration existed, downgrading the system’s security as a side effect of the upgrade.

Finding 3: NES models exhibit *semantic blindness* during transactional edits, prioritizing syntactic fluency over security invariants and producing vulnerability rates of 86% in refactoring and 46.39% in configuration migration.

4.1.3 Human-IDE Interaction: Induced Complacency. Beyond the model itself, the *tab-accept-tab* interaction loop reshapes when and where developers pause to verify a suggestion. Our C3 scenarios reveal two complementary effects: navigation features that route users *around* verification points, and a streak of correct suggestions that lowers scrutiny once a security-critical decision finally appears.

Bypassing Verification. The *tab-by-tab* paradigm encourages rapid navigation and reshapes which code regions a developer actually inspects. NES’s auto-jump (V10) advances the cursor past intermediate sites and skips 90.00% of vulnerabilities that share the same defect as the one just patched, leaving multi-site issues only partially fixed. Conversely, No-Op suggestions (V11) act as an implicit “done” signal: in 43.00% of cases the absence of a follow-up edit is interpreted as task completion even though remediation remains incomplete. Together, these low-friction patterns reroute the developer’s attention away from the very locations where verification is most needed.

Overtrusting in Functionality. A streak of functionally correct suggestions also desensitizes developers to subsequent security-critical decisions. In V12, a sequence of valid non-security edits builds acceptance momentum, and a final insecure suggestion injected into the same flow is accepted in 69.00% of cases. The trust accumulated by repeatedly correct completions transfers to a security-relevant step that would, in isolation, warrant explicit scrutiny.

Finding 4: The NES interaction loop induces complacency by routing developers around verification points and accumulating trust across correct suggestions, yielding a 66.5% bypass rate of in-flow verification and a 69.00% acceptance rate for insecure edits delivered after a correct streak.

4.2 Black-box Validation Results

The black-box validation on commercial IDEs (GitHub Copilot, Zed, Cursor, and Trae) confirms that the security pitfalls observed above are not artifacts of our specific Zeta evaluations but inherent challenges of the NES architecture itself. Comparing the commercial leaders with our white-box baseline reveals three insights aligned with the risk taxonomy.

Model Upgrade vs. Context Contamination (C1). On the inherent-model vector (V1), commercial IDEs achieve a markedly lower vulnerability rate than Zeta (−32.5%), likely attributable to the continuous *black-box* upgrades applied to their underlying models [20], which suppress the generation of insecure code at the source. However, on the context-dependent vectors (V2–V6) the average vulnerability rate is 7.9% *higher* than that of the base model. Commercial IDEs therefore only partially mitigate inherent risks through model upgrades and largely fail to address the risks introduced by extended context.

Finding 5: Commercial models successfully reduce intrinsic vulnerabilities (−32.5%) but offer no protection against risks introduced by extended context.

Security Downgrades in Transactions (C2). For *Transactional Edits*, the gap between commercial tools and our baseline is minimal, with commercial tools even exhibiting slightly higher vulnerability rates on V7 (+5.5%) and V9 (+16.11%). This alignment confirms that semantic blindness is not an artifact of Zeta but a property of the NES paradigm, and the small increase suggests that in real-world development the diversity of combined context sources further exacerbates the risk.

Finding 6: In real-world development scenarios, commercial IDEs show a slight increase (up to 16.11%) in vulnerability rates when diverse context sources are combined.

Trust Traps in UX Optimizations (C3). The *Human-IDE Interaction* results present a sharp contrast driven by user-experience design. Commercial IDEs cut the vulnerability rate of auto-jumping (V10: -37.5%) but inflate that of No-Op suggestions (V11: +39.5%). Through close inspection and debugging, we attribute this divergence to the debouncing strategies equipped in commercial IDEs, which introduce small delays or suppress suggestions to prevent cognitive overload. In V10, the interruption acts as a pseudo safety brake: it breaks the rapid navigation flow and gives users an opportunity to notice and address security issues overlooked by NES. In V11, the same mechanism backfires—premature termination produces silence at critical moments, which users interpret as safe completion rather than transient latency, leading to skipped remediation. Sequential overtrust (V12) similarly increases by 16%, suggesting that smoother UX tends to prioritize productivity over security vigilance.

Finding 7: UX optimizations in commercial IDEs reshape user vigilance unevenly across vectors, reducing some risks (V10: -37.5%) while inducing complacency that drives substantial increases on others (V11: +39.5%, V12: +16%).

5 Online Survey: Developer Perception and Trust

Our in-lab analyses (Section 4) characterize the technical security implications of NES mechanisms, but they cannot speak to whether and how developers actually perceive these risks during real work. We therefore complement them with an online survey that asks two questions: (RQ3) whether the risks identified in our taxonomy are perceived and experienced by software developers in practice, and (RQ4) how the *tab-accept-tab* loop influences their trust, reliance, and code-scrutiny behaviors when interacting with NES suggestions. These self-reports complement our technical evaluation with a user perspective on NES risk perception but do not independently validate the prevalence of individual risk vectors V1–V12.

We distributed the survey to a sampled population engaged in software development across both industry and academic settings, and collected 385 responses including 241 professional developers; the analysis below is based on 269 responses retained after quality filtering. At a high level, a majority of respondents report encountering NES-induced insecure suggestions in practice (RQ3), and the reported scrutiny behaviors indicate that trust in NES is often granted on the basis of visual plausibility rather than explicit security verification (RQ4).

5.1 Online Survey Design

Recruitment. Our target population comprises students and employees with substantial software development involvement. To include both organizational and academic development contexts, we recruited participants through two complementary channels. First, we distributed the survey across a global IT company that employs over 50,000 software developers across offices in North

Table 3: Respondent Profile and Tool Usage Summary

Respondent Profile				
Role	Professional		Student	Hobbyist
	64.4%		23.0%	12.6%
Experience (years)	0–3 (Junior)	3–5 (Medium)	6–10 (Senior)	10+ (Expert)
	36.4%	33.8%	16.7%	13.1%
Primary Field	CS / SE	Security	AI / DS	Other
	42.4%	32.7%	16.0%	8.9%
Tool Adoption and Usage				
AI IDE Adoption	Aware & Used			Unaware
	95.2%			4.8%
IDE Usage	Always	Often	Occasional	Never
	43.5%	32.0%	21.5%	3.0%
NES Usage	Always	Often	Occasional	Never
	21.2%	35.7%	34.9%	8.2%
NES Dependence	High	Medium	Low	
	16.4%	43.1%	40.5%	

Distribution of respondent demographics and their adoption and usage of AI-integrated IDEs. **DS:** Data Science; **SE:** Software Engineering

America, Europe, and the Asia-Pacific region, inviting respondents from a wide range of teams, products, and engineering domains rather than a single business unit. The company permits employees to use several AI-integrated IDEs for non-confidential projects, including GitHub Copilot, Cursor, Zed, and an internal AI IDE. This list reflects company policy rather than participant-level usage and we did not collect the corresponding usage distribution because of company policy restrictions. Second, we posted the survey on the advertisement platform of a U.S. top-100 university to recruit post-graduate students in computer science. Together, these channels provide organizational and academic perspectives, and the resulting sample includes participants with varied levels of development experience.

In both channels, participants accessed the survey via a secure online platform. A consent form at the beginning outlined the study purpose, estimated time cost (< 5min), and the measures taken for anonymity and data confidentiality. Participants who completed the survey received compensation aligned with 10% of the average hourly wage of software developers in their respective location [17], a common practice for sustaining participation rates and response quality [51].

Structure of the Survey. To understand the gap between prevalent NES usage and the limited perception of NES risks among general AI-IDE users who are unlikely to know the internal components of NES, we designed our survey grounded on prior literature [46, 48] and to avoid specific NES components that would reduce generalizability and bias the study population, with 12 questions constructed into three main sections: (1) Demographic Information, (2) NES Usage Patterns, (3) Trust and Security Perceptions. Demographic information was collected to understand the background of the participants, including their experience level, occupation, and professional domain. The NES Usage Patterns section focused on the participants' familiarity and frequency of using NES features, and

their dependency degree on NES. The Trust and Security Perceptions section aimed to assess participants’ trust levels in NES suggestions, their awareness of potential security pitfalls, and any security incidents they may have encountered while using NES features. Additionally, an Instructional Manipulation Check (IMC) [42] was included in our survey to verify that questions were answered with attention. The survey can be accessed via <https://github.com/plainoodle-expert/NesCodeSec/blob/main/Questionnaire.pdf>.

5.2 Demographics

We initially collected 385 responses from the online survey. After the rigorous data cleaning process by IMC, we finalized a dataset of 269 high-quality responses for analysis. Table 3 shows the demographic profile of the 269 respondents.

Roles. The study’s participant pool is predominantly composed of professional developers (64.4%), ensuring that the findings reflect the complexities and requirements of industrial software development. This core demographic is supplemented by students (23%) and hobbyists (12.6%), representing the emerging workforce and the part-time developer community, respectively.

Experience. The participant sample is characterized by a significant concentration of experienced developers, providing the requisite expertise to analyze the security risks associated with NES systems. A majority of respondents (63.6%) possess more than three years of professional software development experience. Notably, 29.8% of the participants are senior or expert developers with over six years of experience, contributing deep technical insight. This core demographic is complemented by junior developers (< 3 years), who constitute 36.4% of the population. This distribution, while dominated by seasoned professionals, retains a representative cohort of early-career practitioners, facilitating a nuanced examination of how experience levels influence the perception and trust of NES security risks.

Field. Participants were recruited from diverse technical fields, with a predominant focus on computer science / software engineering (42.4%) and cyber security (32.7%), alongside a significant cohort from AI / data science (16%). This distribution is methodologically critical: **the strong representation of security professionals provides a high-competence baseline for perceiving subtle vulnerabilities**, while the mix of generalist developers and AI specialists allows us to assess whether domain-specific knowledge (e.g., security) influences their trust to vulnerable NES suggestions.

5.3 AI-Integrated IDE Ecosystem

The survey results indicate that AI-integrated tools have achieved near-universal adoption among the surveyed population, while developer reliance on them is widespread but unevenly distributed across usage patterns.

Adoption and Frequency. As reported in Table 3, AI-integrated IDEs are nearly ubiquitous: 95.2% of respondents use them, and 75.5% (43.5% *always* + 32.0% *often*) engage with these tools as a regular part of their development workflow. At this level of usage, AI-integrated IDEs are no longer a peripheral productivity aid but a routine component of how code is written, which in turn means that any inherent vulnerabilities propagate at the scale of the developer ecosystem rather than to a niche subset.

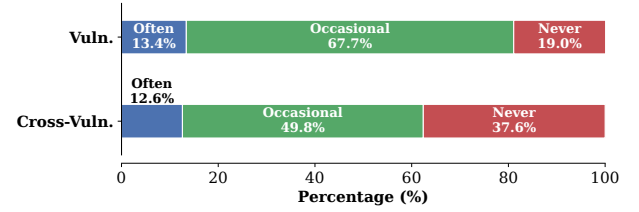


Figure 5: Developer perceptions of NES risks: (a) insecure suggestions and (b) test-to-production data leakage.

NES Engagement vs. Dependency. A majority of participants (56.9%) use NES features frequently, and 59.5% report medium-to-high dependency on NES, indicating that for most respondents NES has moved beyond an auxiliary convenience and is regularly relied upon during coding. This level of integration amplifies the impact of any insecure suggestion, making the security analysis in Section 4 operationally relevant rather than purely theoretical. The dependency is further skewed toward professionals: 62.4% of professional developers report medium or high reliance, compared to 54.2% among students and hobbyists. A plausible explanation is that professionals routinely face a large volume of complex and business-critical engineering tasks, and therefore turn to NES as a fine-grained efficiency lever that keeps them in control of each edit, rather than forfeiting authorship of the resulting code by using more automated code generation feature in IDEs.

5.4 Perception of Security Risks in NES

As shown in Figure 5, more than 80% of participants report having encountered insecure suggestions generated by NES, indicating that exposure to NES-induced risks is prevalent across the surveyed population rather than confined to isolated cases. Yet self-reported awareness of these risks and the ability to identify insecure suggestions vary substantially across respondents, pointing to a gap between *having seen* insecure suggestions and *being able to recognize* them in the moment of acceptance.

Awareness of NES-Induced Risks. Developers report encountering, in practice, the same classes of vulnerabilities our technical analysis surfaces in the lab, suggesting that these risks are not merely theoretical artifacts of controlled test cases. As shown in Figure 5, 81.1% of respondents report observing insecure code suggestions from NES, and 13.4% report frequent exposure (> 10 times daily). More critically, 62.4% report witnessing NES propose test-environment data (e.g., hardcoded keys) for production use—a self-reported manifestation of the cross-context leakage we characterize as *Context Contamination* in Section 4. Taken together, these responses align with the context-mixing behaviors observed in our assessments and indicate a concrete pathway by which such suggestions can introduce vulnerabilities if accepted without scrutiny.

Security Experience vs. Awareness. Despite the prevalent exposure to risky suggestions reported above, awareness and detection are unevenly distributed across participants, and neither a cybersecurity background nor years of experience alone appears sufficient to close this gap. We first examined whether a cybersecurity background correlates with greater vigilance. A Spearman rank

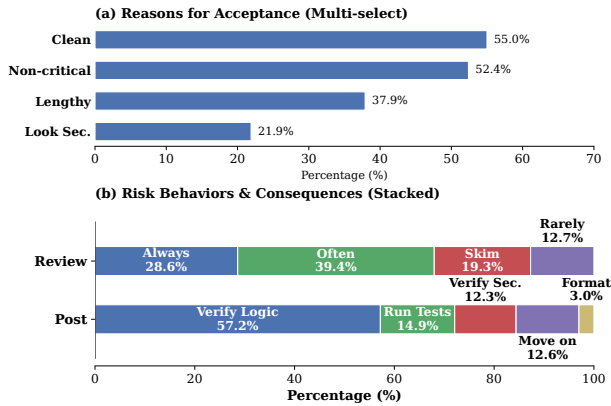


Figure 6: The trust of NES suggestions among developers.

correlation between participants’ primary field (non-cybersecurity = 1, cybersecurity = 2) and self-reported code review behavior (never = 1, always = 5) yields only a weak positive correlation, indicating that domain training is at best modestly associated with more frequent review and does not, on its own, translate into reliably stronger detection. Development experience tells a similar story: 37.5% of respondents report never encountering sensitive data leakage in production contexts, and within this “unnoticing” cohort security practitioners account for 24.8%—below their 32.7% share in the overall sample—while 60% of them are junior developers, well above the 36.4% junior representation in the full sample. The two factors appear to reinforce each other rather than substitute. Junior security practitioners are overrepresented in the unnoticing cohort, even though they have security training. This suggests two complementary points: security *knowledge* helps only when developers have enough hands-on experience to spot it being violated in real suggestions; and experience alone, without security awareness, rarely surfaces security-specific defects. In practice, lowering NES risk requires both, not either one alone.

Finding 8: Detecting NES-induced risks imposes a high competence bar: 81.1% of developers report exposure, but only those with *both* a cybersecurity background and substantial development experience reliably turn exposure into detection.

5.5 Trust in NES Suggestions

The survey reveals a disconnect between the prevalence of NES-induced risks and the rigor of developer verification. As shown in Figure 6, security is often assumed rather than explicitly validated.

Scrutiny Bias. Only 28.6% of respondents report ‘always’ thoroughly reviewing NES-generated code, while 32.0% admit to only ‘skim’ or ‘rarely’ scrutinizing the output. This lack of scrutiny is often driven by visual plausibility rather than semantic correctness: 55.0% accept suggestions because the code looks ‘clean’, and 37.9% accept unreviewed code to avoid typing complex or ‘lengthy’ constructs. This behavior aligns with our analysis that cognitive load drives security compromises.

Mitigation Absence. The verification effort developers do invest is overwhelmingly directed at functional correctness rather

than security. While 57.2% prioritize validating the *functional logic* of suggested code, only 12.3% explicitly check *security aspects*, and a further 14.9% rely solely on standard test execution—a pipeline well suited to functional bugs but largely blind to security issues such as logical bypasses or data leakage. The side effect is that, even when NES output is reviewed, the review rarely targets the failure modes our in-lab analysis identifies, leaving security properties implicitly delegated to the model rather than verified by the developer.

Finding 9: Verification of NES suggestions is mostly assumed, not enacted: 28.6% consistently review NES output and only 12.3% check security—so security is largely left to the model.

6 Discussion

Our mechanism-driven evaluation shows that NES security failures can arise from the underlying model and be amplified by context construction, multi-step edit prediction, and low-friction interaction. Our survey complements these results with self-reported evidence on respondents’ awareness, trust, and verification practices. We next discuss the implications and limitations.

6.1 Mitigating NES Security Risks

The cross-system results suggest that improving the underlying model is necessary but insufficient: commercial systems reduced inherent-model failures, yet did not consistently prevent manifestations activated by extended context, transactional edits, and interaction patterns (Section 4.2). Although mature NES defenses remain limited, our findings suggest several concrete mitigations.

For AI-IDE providers. The context contamination results (V2–V6) motivate explicit context provenance and user control. AI-IDE providers should clearly document the source of context that will be gathered for suggestion generation, rather than relying on implicit assumptions about developer awareness [12]. A mechanism should also provide users with the ability to review, filter, or exclude context from untrusted sources, such as `.cursorignore` [11]. Second, the semantic blindness of underlying models in transactional edits (V7–V9) motivates strengthening models’ ability to understand and preserve security semantics when generating edits. Providers can incorporate security-oriented prompting [53], security-aware instruction tuning [28], and proactive security alignment [55], which prior work has shown can improve secure code generation. Third, the reduced vigilance in the interaction results (V10–V12) suggests that users may skip scrutiny due to increased cognitive load. Providers should automatically assess each generated suggestion for potential security implications and present the review result alongside the suggestion before acceptance, rather than delegating the reviews entirely to users.

For developers. First, organizations should strengthen training in secure coding and reviewing AI-generated edits [18, 47]. Our survey findings in Section 5.4 suggest that security knowledge is most useful when paired with hands-on experience. Second, developers should treat NES suggestions as unverified code and inspect the complete edit transaction, not only the highlighted edit. Third, developers should incorporate automated security-analysis tools and IDE integrations into their workflows, such as SonarQube for IDE [50] and Checkmarx Developer Assistant [6], to identify

potential issues without treating them as substitutes for manual review. Given that only 12.3% of respondents reported explicitly checking suggestion security, these measures may address the observed verification gap, although their effectiveness in practical NES workflows remains to be evaluated.

6.2 Broader Implications and Future Directions

Several manifestations overlap with vulnerability classes reported for LLM-based coding tools [38, 45], but not necessarily through identical causal pathways. Our results distinguish model-inherited vulnerabilities from those mediated or amplified by implicit context aggregation, multi-step edit prediction, and low-friction acceptance. The security boundary thus extends beyond the generated snippet to the context-to-edit workflow.

Repository-scale context and multi-step modification also appear in agentic coding systems [2, 24, 41]. However, NES continuously infers intent and presents proactive edits for per-suggestion acceptance, whereas agents may plan, invoke tools, execute code, and modify multiple files. Our context and transactional mechanisms therefore provide hypotheses for evaluating agents, although tab-driven interaction effects may not transfer directly. Agents also introduce execution and permission risks outside our taxonomy [16]. Future work should test whether these pathways and mitigations generalize to agentic tools and broader development settings.

6.3 Threats to Validity

Technical scope and measurement. Our evaluation is limited to Java artifacts, one open-source NES model, and four commercial AI-integrated IDEs; manifestations in Table 1 may differ across languages, ecosystems, and future systems. Because our scenarios deliberately activate each pathway, reported rates are conditional failure rates, not prevalence estimates for ordinary development. LLM judges complement rule-based evaluation and may be prompt- or model-sensitive. A strong-majority rule and manual review reduce, but cannot eliminate, misclassification.

Participant sample and survey measurement. We recruited participants through one multinational IT company and one university, with a substantial portion of respondents coming from the company. Developers recruited through the company may be more likely to receive security training and follow shared organizational practices, and their self-reported behaviors may therefore differ from those of independently recruited or more loosely affiliated developers. At the same time, this access provides an enterprise-embedded perspective on developers' awareness of emerging security risks, complementing prior studies of independently recruited or more loosely affiliated developers [3, 46, 48], though this perspective may not generalize across organizations. Voluntary responses are also susceptible to self-selection, social-desirability, and interpretation biases. The IMC reduces inattentive responses but does not address these broader biases.

IDE information and scope of claims. The company's permitted IDEs provide policy context, not participant-level evidence of tool use. Without participant-level IDE identities, we cannot compare practices or attribute reported behaviors or reduced vigilance to specific tools. The survey supports conclusions about awareness,

trust, and reported verification within the sampled cohorts, not the prevalence of V1–V12, objective detection accuracy, or population-wide AI-IDE behavior.

7 Related Work

We trace developer assistance from program-analysis-based IDE services to NES and agentic coding, then synthesize security research on model outputs, developer interaction, and adversarial manipulation to clarify the gap addressed by our work.

7.1 Evolution of AI-Assisted Software Development

Traditional IDEs provided deterministic assistance through program analysis over representations such as Abstract Syntax Trees (ASTs) [35] and language services exposed through the Language Server Protocol (LSP) [37]. LLMs extended this assistance to probabilistic generation and context-aware suggestions spanning the editing location and broader project context [58].

Building on this shift, Next-Edit Suggestion (NES) systems [8], as implemented in modern AI-assisted IDEs [9, 10, 56], move beyond predicting an immediate continuation at the cursor. NES instead analyzes developers' ongoing editing activities together with broader project context to proactively predict subsequent code modifications, shifting AI assistance from reactive completion to proactive edit prediction across multiple locations or larger semantic changes.

Beyond proactive edit prediction, AI-assisted software development is also moving toward increasingly autonomous coding agents, such as Claude Code [2], OpenAI Codex [41], and GitHub Copilot Agent [24]. These systems extend assistance beyond code suggestions to iterative workflows that plan, execute, test, and revise code. Despite this evolution, proactive context-aware editing remains an important capability of modern AI-integrated development environments.

7.2 Security of AI-Assisted Software Development

Against this evolution, existing studies have extensively investigated the security of AI-generated code. Pearce et al. found that GitHub Copilot frequently produced insecure snippets in controlled security scenarios [45], while subsequent work showed that LLMs can reproduce insecure coding practices or fail to generate secure-by-default implementations [30]. Moving beyond controlled generation settings, Fu et al. identified diverse security weaknesses in AI-generated snippets collected from public projects [19]. Collectively, these studies characterize insecurity at the model-output level across both constructed scenarios and deployed codebases.

Security outcomes, however, are not determined by model outputs alone. Controlled user studies have examined how access to coding assistants, prompting behavior, trust, and verification practices influence security outcomes [46, 48]. Klemmer et al. further characterized how software professionals use AI assistants in security-relevant tasks and review their suggestions in practice [31]. These studies establish developer interaction as a critical factor in AI-assisted software security. Our survey complements them by examining developers' awareness of NES-related risks and their reported scrutiny of proactively generated edits.

A separate line of work examines adversarial manipulation. Poisoning can steer code models and completion systems toward insecure patterns [40, 49], while indirect prompt injection can manipulate LLM-integrated applications through untrusted context [26]. With greater autonomy, repository access, and tool use, consequences may extend to file modification, command execution, and multi-step actions [36]. These studies expose risks in model behavior, contextual inputs, and autonomous execution.

Taken together, prior studies primarily examine insecurely generated snippets, developers' handling of explicitly requested assistant output, or adversarial risks in tool-enabled agents. In contrast, NES continuously observes an ongoing edit stream, implicitly assembles context from the development environment, and proactively presents predicted modifications for low-friction acceptance. Our work complements prior research by systematically connecting these architectural and interaction mechanisms to security failures arising from implicit context propagation, transactional edits, and human–IDE interaction. Because related context-aware editing capabilities also appear in agentic workflows, these connections further motivate future research on whether the identified risks transfer to more autonomous coding systems.

8 Conclusion

In this paper, we presented the first systematic security analysis of NES systems. Our dissection of NES architectures revealed 12 novel threat vectors exploitable through imperceptible developer actions. Both commercial and open-source NES implementations exhibit an average over 70% vulnerability rate under context contamination and manipulation. The user study reveals a gap in awareness and verification of associated security risks. These findings emphasize the urgent need for security-aware design patterns and automated defenses in future AI-assisted programming environments.

Acknowledgment

We thank the anonymous reviewers for their valuable feedback. This work was supported by The Hong Kong Jockey Club.

References

- [1] Bonnie Brinton Anderson, C. Brock Kirwan, Jeffrey L. Jenkins, David Eargle, Seth Howard, and Anthony Vance. 2015. How Polymorphic Warnings Reduce Habituation in the Brain: Insights from an fMRI Study. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, 2883–2892. doi:10.1145/2702123.2702322
- [2] Anthropic. 2026. *How Claude Code Works*. Retrieved August 3, 2026 from <https://code.claude.com/docs/en/how-claude-code-works>
- [3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030
- [4] Manish Bhatt et al. 2023. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. arXiv:2312.04724 [cs.CR] <https://arxiv.org/abs/2312.04724>
- [5] Ruisheng Cao et al. 2026. Qwen3-Coder-Next Technical Report.
- [6] CheckmarkX. 2026. *CheckmarkX Developer Assistant*. Retrieved August 1, 2026 from <https://checkmarx.com/product/developer-assist>
- [7] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [8] Xinfang Chen, Siyang Xiao, Xianying Zhu, Junhong Xie, Ming Liang, Dajun Chen, Wei Jiang, Yong Li, and Peng Di. 2025. An Efficient and Adaptive Next Edit Suggestion Framework with Zero Human Instructions in IDEs. arXiv:2508.02473 [cs.SE] <https://arxiv.org/abs/2508.02473>
- [9] Cognition, Inc. 2024. *Windsurf - The best AI for Coding*. Retrieved August 1, 2026 from <https://windsurf.com/>
- [10] Cursor. 2024. *Cursor IDE*. Retrieved August 1, 2026 from <https://cursor.com/>
- [11] Cursor. 2025. *Cursor Ignore File*. Retrieved August 1, 2026 from <https://cursor.com/cn/docs/reference/ignore-file>
- [12] Cursor. 2025. *Cursor Issue: Cursor AI View Code Leak*. Retrieved August 1, 2026 from <https://forum.cursor.com/t/big-security-risk-cursorignore-doesnt-seem-to-work-envs-files-being-sent-as-context/14027>
- [13] Cursor. 2025. *Cursor Issue: Cursor keeps trying to access sensitive env variables even though .env is ignored*. Retrieved August 1, 2026 from <https://forum.cursor.com/t/cursor-keeps-trying-to-access-sensitive-env-variables-even-though-env-is-ignored/145607>
- [14] Cursor. 2025. *Cursor Issue: Cursor reads .env even though it is on .cursorignore*. Retrieved August 1, 2026 from <https://forum.cursor.com/t/cursor-reads-env-even-though-it-is-on-cursorignore/136998>
- [15] Cursor. 2025. *Questions on .gitignore, .cursorignore, .cursorban*. Retrieved August 1, 2026 from <https://forum.cursor.com/t/questions-on-gitignore-cursorignore-cursorban/34713>
- [16] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2636, 26 pages.
- [17] Neal Dickert and Christine Grady. 1999. What's the price of a research subject? Approaches to payment for research participation. *The New England journal of medicine* 341, 3 (1999), 198–203. <https://api.semanticscholar.org/CorpusID:45204547>
- [18] Tiago Espinha Gasiba, Ulrike Lechner, Maria Pinto-Albuquerque, and Daniel Méndez. 2021. Is Secure Coding Education in the Industry Needed? An Investigation Through a Large Scale Survey. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE Press, Virtual Event, Spain, 241–252. doi:10.1109/ICSE-SEET52601.2021.00034
- [19] Yujia Fu, Peng Liang, Amjed Tahir, Zengyang Li, Mojtaba Shahin, Jiaxin Yu, and Jinfu Chen. 2025. Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Trans. Softw. Eng. Methodol.* 34, 8, Article 218 (Oct. 2025), 34 pages. doi:10.1145/3716848
- [20] GitHub. . 2025. *Evolving GitHub Copilot's next edit suggestions through custom model training*. Retrieved August 1, 2026 from <https://github.blog/ai-and-ml/github-copilot/evolving-github-copilots-next-edit-suggestions-through-custom-model-training/>
- [21] GitHub, Inc. 2021. *Code Scanning with CodeQL*. Retrieved August 1, 2026 from <https://codeql.github.com/>
- [22] GitHub, Inc. 2024. *GitHub Copilot · Your AI pair programmer*. Retrieved August 1, 2026 from <https://github.com/features/copilot>
- [23] GitHub, Inc. 2025. *Copilot Next Edit Suggestions (preview)*. Retrieved August 1, 2026 from <https://code.visualstudio.com/blogs/2025/02/12/next-edit-suggestions>
- [24] GitHub, Inc. 2025. *Introducing GitHub Copilot agent mode (preview)*. Retrieved August 1, 2026 from <https://code.visualstudio.com/blogs/2025/02/24/introducing-copilot-agent-mode>
- [25] GitHub, Inc. 2026. *Tree-sitter: An Incremental Parsing System for Programming Tools*. Retrieved August 1, 2026 from <https://github.com/tree-sitter/tree-sitter>
- [26] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (Copenhagen, Denmark) (AISec '23)*. Association for Computing Machinery, New York, NY, USA, 79–90. doi:10.1145/3605764.3623985
- [27] Daya Guo et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] <https://arxiv.org/abs/2401.14196>
- [28] Jingxuan He, Mark Vero, Gabriela Krasnopolska, and Martin Vechev. 2024. Instruction tuning for secure code generation. In *Proceedings of the 41st International Conference on Machine Learning (ICML '24)*. JMLR.org, Vienna, Austria, Article 723, 20 pages.
- [29] Binyuan Hui et al. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL] <https://arxiv.org/abs/2409.12186>
- [30] Raphaël Khoury, Anderson R. Avila, Jacob Brunelle, and Baba Mamadou Camara. 2023. How Secure is Code Generated by ChatGPT? arXiv:2304.09655 [cs.CR] <https://arxiv.org/abs/2304.09655>
- [31] Jan H. Klemmer et al. 2024. Using AI Assistants in Software Development: A Qualitative Study on Security Practices and Concerns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (Salt Lake City, UT, USA) (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 2726–2740. doi:10.1145/3658644.3690283
- [32] Raymond Li et al. 2023. StarCoder: may the source be with you! *Transactions on Machine Learning Research* 2 (2023), 73 pages. <https://openreview.net/forum?id=KoF0g41haE>

- [33] Yujia Li et al. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. doi:10.1126/science.abq1158
- [34] J. Lin. 1991. Divergence measures based on the Shannon entropy. *IEEE Transactions on Information Theory* 37, 1 (1991), 145–151. doi:10.1109/18.61115
- [35] Chris J. Maddison and Daniel Tarlow. 2014. Structured generative models of natural source code. In *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32 (ICML '14)*. PMLR, Beijing, China, 649–657.
- [36] Narek Maloyan and Dmitry Namiot. 2026. Prompt Injection Attacks on Agentic Coding Assistants: A Systematic Analysis of Vulnerabilities in Skills, Tools, and Protocol Ecosystems. arXiv:2601.17548 [cs.CR] <https://arxiv.org/abs/2601.17548>
- [37] Microsoft. 2016. *Language Server Protocol*. Retrieved August 1, 2026 from <https://microsoft.github.io/language-server-protocol/>
- [38] Yutao Mou, Xiao Deng, Yuxiao Luo, Shikun Zhang, and Wei Ye. 2025. Can You Really Trust Code Copilots? Evaluating Large Language Models from a Code Security Perspective. arXiv:2505.10494 [cs.CL] <https://arxiv.org/abs/2505.10494>
- [39] Robert C Nickerson, Upkar Varshney, and Jan Muntermann. 2013. A method for taxonomy development and its application in information systems. *European journal of information systems* 22, 3 (2013), 336–359.
- [40] Sanghak Oh, Kiho Lee, Seonhye Park, Doowon Kim, and Hyoungshick Kim. 2024. Poisoned ChatGPT Finds Work for Idle Hands: Exploring Developers’ Coding Practices with Insecure Suggestions from Poisoned AI Models. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1141–1159. <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00046>
- [41] OpenAI. 2025. *Introducing Codex*. Retrieved August 1, 2026 from <https://openai.com/index/introducing-codex/>
- [42] Daniel M. Oppenheimer, Tom Meyvis, and Nicolas Davidenko. 2009. Instructional manipulation checks: Detecting satisficing to increase statistical power. *Journal of Experimental Social Psychology* 45, 4 (2009), 867–872. doi:10.1016/j.jesp.2009.03.009
- [43] OWASP Foundation. 2024. *XML External Entity (XXE) Processing*. OWASP. Retrieved August 1, 2026 from [https://owasp.org/www-community/vulnerabilities/XML_External_Entity_\(XXE\)_Processing](https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing)
- [44] OWASP Foundation. 2025. *OWASP Top 10 - 2025*. OWASP. Retrieved August 1, 2026 from <https://owasp.org/Top10/2025/>
- [45] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, IEEE Computer Society, Los Alamitos, CA, USA, 754–768. doi:10.1109/SP46214.2022.9833571
- [46] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do Users Write More Insecure Code with AI Assistants?. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 2785–2799. doi:10.1145/3576915.3623157
- [47] Julia Prümmer, Tommy van Steen, and Bibi van den Berg. 2024. A systematic review of current cybersecurity training methods. *Comput. Secur.* 136, C (Jan. 2024), 20 pages. doi:10.1016/j.cose.2023.103585
- [48] Gustavo Sandoval, Hammond Pearce, Teo Nys, Ramesh Karri, Siddharth Garg, and Brendan Dolan-Gavitt. 2023. Lost at C: a user study on the security implications of large language model code assistants. In *Proceedings of the 32nd USENIX Conference on Security Symposium*. USENIX Association, Anaheim, CA, USA, 2205–2222. <https://dl.acm.org/doi/10.5555/3620237.3620361>
- [49] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. 2021. You Autocomplete Me: Poisoning Vulnerabilities in Neural Code Completion. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Vancouver, B.C., Canada, 1559–1575. <https://www.usenix.org/conference/usenixsecurity21/presentation/schuster>
- [50] SonarSource. 2026. *SonarQube for IDE*. Retrieved August 1, 2026 from <https://www.sonarsource.com/products/sonarqube/ide/>
- [51] Guy Stecklov, Alex Weinreb, and Gero Carletto. 2017. Can Incentives Improve Survey Data Quality in Developing Countries?: Results from a Field Experiment in India. *Journal of the Royal Statistical Society: Series A (Statistics in Society)* 181, 4 (2017), 1033–1056. doi:10.1111/rssa.12333
- [52] Alexey Svyatkovskiy, Ying Zhao, Shengyu Fu, and Neel Sundaresan. 2019. Pythia: AI-assisted Code Completion System. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Anchorage, AK, USA) (KDD '19)*. Association for Computing Machinery, New York, NY, USA, 2727–2735. doi:10.1145/3292500.3330699
- [53] Catherine Tony, Nicolás E. Díaz Ferreyra, Markus Mutas, Salem Dhif, and Ricardo Scandariato. 2025. Prompting Techniques for Secure Code Generation: A Systematic Investigation. *ACM Trans. Softw. Eng. Methodol.* 34, 8, Article 225 (Oct. 2025), 53 pages. doi:10.1145/3722108
- [54] TRAE. 2025. *TRAE: The Real AI Engineer*. Retrieved August 1, 2026 from <https://www.trae.ai/>
- [55] Xiangzhe Xu, Zian Su, Jinyao Guo, Kaiyuan Zhang, Zhenting Wang, and Xiangyu Zhang. 2025. PROSEC: fortifying code LLMs with proactive security alignment. In *Proceedings of the 42nd International Conference on Machine Learning (ICML '25)*. JMLR.org, Vancouver, Canada, Article 2784, 16 pages.
- [56] Zed Industries. 2024. *Zed - Code at the speed of thought*. Retrieved August 1, 2026 from <https://github.com/zed-industries/zed>
- [57] Zed Industries. 2024. *Zeta*. Retrieved August 1, 2026 from <https://huggingface.co/zed-industries/zeta>
- [58] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151

A Open Science

To comply with Open Science Policy and to support reproducibility and independent evaluation, we make available all artifacts necessary to understand, assess, and replicate the core findings of this work, subject to ethical and security considerations. The artifacts are accessible from: <https://github.com/plain-noodle-expert/NesCodeSec>.

B Ethical Considerations

We obtained IRB approval before conducting our survey and distributed the survey on reputable and reliable platforms. During the survey, we ensured all participants were provided informed consent, and participation was entirely voluntary, and they could withdraw at any time. The survey consists of no sensitive questions or questions that may disclose personal information. The number of questions is designed to cost approximately 5 minutes for participants. To recognize their time and effort, we offered a monetary compensation to each participant upon survey completion.

C Detection of Insecure NES Suggestions

For each test case, we retain the original code and the generated NES suggestion. Because a suggestion may contain only a partial diff, we first apply it to the original file to reconstruct the post-edit program. We then evaluate the prespecified criterion for each vector using regex-based scanning for structural patterns (e.g., MD5 or hardcoded credentials), AST-based static analysis for syntactic and configuration properties (e.g., XML parser settings), or LLM-based semantic analysis for risks requiring data-flow or intent reasoning (e.g., sensitive logging or intended Undo actions). Unrelated weaknesses are not counted toward that vector. For the LLM evaluated cases, we use a five-model panel comprising DeepSeek-V3.2, Qwen3-235B-A22B, Gemini-3-Flash, Claude-Haiku-4.5, and GPT-5-mini. Each model is queried separately in a fresh session using the same vector-specific prompt and has no access to the responses of the other judges. The input contains the relevant pre-edit code or interaction trace, the NES suggestion, the reconstructed post-edit code, and the security criterion for the target manifestation. Each judge independently returns a binary label, where 1 indicates that the target manifestation is present and 0 indicates that it is absent. We use a symmetric strong-majority rule: an output is automatically labeled vulnerable for the target vector when at least four judges return 1, and not vulnerable for that vector when at least four return 0. The produced ambiguous cases (2 vs 3 votes), totaling 51 samples in our evaluation, are independently reviewed by the authors. The resulting adjusted label is used in the final vulnerability-rate calculation.